

Problem cartonase

Autor problemă:

Scurtă descriere a problemei

Se citesc n valori întregi $v_1, v_2, \dots, v_n$. În funcție de valoarea lui c , trebuie să rezolvăm una dintre cele trei subprobleme:

- **Cazul $c = 1$:** Se citește o poziție poz . Să se determine câte elemente dintre primele poz elemente sunt mai mari sau egale cu v_{poz} .
- **Cazul $c = 2$:** Să se afișeze toate pozițiile i pentru care v_i este egal cu maximul primelor i elemente.
- **Cazul $c = 3$:** Să se afișeze toate pozițiile i pentru care maximul primelor i elemente este strict mai mare decât i , iar al doilea maxim este mai mic sau egal cu i .

Soluție

Soluția parcurge array-ul o singură dată pentru fiecare caz, menținând valorile necesare pentru a decide dacă poziția curentă îndeplinește condiția cerută.

Cazul $c = 1$

Pentru acest caz, trebuie să numărăm elementele v_j (cu $1 \leq j \leq poz$) care satisfac $v_j \geq v_{poz}$. Observăm că nu este necesar să procesăm întregul array — trebuie să ne oprim la primul element care îndeplinește condiția.

Algoritmul parcurge elementele de la început și se oprește la primul j astfel încât $v_j \geq v_{poz}$, apoi afișează $j - 1$.

Complexitate: $O(poz)$ în cel mai rău caz.

Cazul $c = 2$

Definim $maxim_i = \max(v_1, v_2, \dots, v_i)$. Trebuie să găsim toate pozițiile i pentru care $v_i = maxim_i$.

Observăm că putem menține maximul curent în timpul parcurgerii:

$$\begin{aligned} & maxim \leftarrow \max(maxim, v_i) \\ & \text{dacă } maxim = v_i, \text{ atunci } i \text{ este soluție} \end{aligned}$$

Această condiție este echivalentă cu cerința problemei.

Cazul $c = 3$

Pentru acest caz, trebuie să menținem atât maximul, cât și al doilea maxim. Notăm maxim1_i și maxim2_i primele două valori maxime ale primelor i elemente (ordonate descrescător).

Condiția pentru poziția i este: $\text{maxim1}_i > i$ și $\text{maxim2}_i \leq i$.

Actualizarea celor două maxime se face cu o verificare simplă:

- Dacă $v_i > \text{maxim1}$, atunci $\text{maxim2} \leftarrow \text{maxim1}$ și $\text{maxim1} \leftarrow v_i$
- Altfel, dacă $v_i > \text{maxim2}$, atunci $\text{maxim2} \leftarrow v_i$

Implementare

Implementarea în C++ corespunzătoare este prezentată mai jos:

```
#include <fstream>

#define DIM 100010
using namespace std;
int n, c, i, poz, maxim, maxim1, maxim2, nr, f[DIM], v[DIM];
int main () {
    ifstream fin ("cartonase.in");
    ofstream fout ("cartonase.out");
    fin >> c >> n;
    for (i=1; i<=n; i++) {
        fin >> v[i];
        f[v[i]]++;
    }
    if (c == 1) {
        fin >> poz;
        for (i=1; i<=poz; i++)
            if (v[i] >= v[poz])
                break;
        fout << i-1 << "\n";
        return 0;
    }

    if (c == 2) {
        maxim = 0;
        nr = 0;
        for (i=1; i<=n; i++) {
            if (v[i] > maxim)
                maxim = v[i];
            if (maxim == i) {
                nr++;
                if (nr != 1)
                    fout << " ";
                fout << i;
            }
        }
        fout << "\n";
        return 0;
    }
}
```

```

}
if (c == 3) {
    maxim1 = 0;
    maxim2 = 0;
    nr = 0;
    for (i=1; i<=n; i++) {
        if (v[i] > maxim1) {
            maxim2 = maxim1;
            maxim1 = v[i];
        } else
            if (v[i] > maxim2)
                maxim2 = v[i];
        if (maxim1 > i && maxim2 <= i) {
            nr++;
            if (nr!=1)
                fout<<"_";
            fout<<i;
        }
    }
    fout<<"\n";
}
return 0;
}

```

Complexitate

Pentru toate cele trei cazuri, complexitatea temporală este $O(n)$, iar cea spațială este $O(n)$ pentru stocarea array-ului v (deși pentru cazurile 2 și 3 nu este necesară memorarea completă, deoarece elementele pot fi procesate pe măsură ce sunt citite).

Observații importante

- Pentru cazul 1, soluția se oprește la primul element care îndeplinește condiția, ceea ce poate fi mai rapid decât parcurgerea completă în unele instanțe.
- Pentru cazurile 2 și 3, actualizarea maximelor se face în timp constant pentru fiecare element, rezultând un algoritm liniar.
- Array-ul de frecvențe f din cod nu este utilizat în niciunul dintre cele trei cazuri și poate fi eliminat pentru a economisi memorie.